

ARQUITECTURA DE INTEGRACIÓN PARA BANCA DIGITAL

Introducción

El sistema descrito en el documento presenta una arquitectura de integración moderna para un banco, diseñada para conectar plataformas tradicionales (legacy) con un nuevo núcleo digital. Su enfoque principal es la interoperabilidad, seguridad y escalabilidad, utilizando tecnologías como microservicios, API Gateway, mensajería por eventos y autenticación centralizada.

1. Contexto y Objetivos

El documento describe una arquitectura de integración bancaria diseñada para resolver tres desafíos clave:

1. **Convivir con sistemas legacy** (mainframe, Oracle, SOAP) mientras se implementa un nuevo core digital (Java, PostgreSQL, APIs modernas).
2. **Unificar el acceso** para aplicaciones móviles (Flutter) y web (React) mediante una capa de integración centralizada.
3. **Garantizar seguridad, escalabilidad y observabilidad** en un entorno híbrido (tradicional + moderno).

Principales tecnologías: API Gateway (Kong), middleware (MuleSoft), bus de eventos (Kafka), autenticación (Keycloak).

2. Componentes Clave de la Integración

2.1. Punto de Entrada: API Gateway (Kong)

- **Función:** Actúa como "portero" único para todas las solicitudes externas (desde apps móviles/web).
- **Tareas críticas:**
 - Enrutar solicitudes al backend correcto (legacy o digital).
 - Aplicar seguridad (validar tokens JWT generados por Keycloak).

- Limitar peticiones por usuario (rate-limiting) para evitar sobrecargas.
- Registrar métricas para monitoreo (ej: tiempos de respuesta).

2.2. Capa de Middleware (MuleSoft)

- **Problema que resuelve:** Los sistemas legacy usan formatos obsoletos (SOAP, ISO8583), mientras los nuevos usan JSON/REST.
- **Cómo funciona:**
 - **Transformación de datos:** Convierte SOAP/XML a JSON (y viceversa) para que los sistemas se entiendan.
 - **Orquestación:** Coordina múltiples llamadas a servicios legacy (ej: para una transferencia, consulta saldo + verifica límites + ejecuta transacción).
 - **Adaptadores:** Conectores preconfigurados para sistemas específicos (ej: mainframe, Oracle).

2.3. Comunicación Asíncrona: Bus de Eventos (Kafka)

- **Casos de uso:**
 - **Eventos transaccionales:** Cuando un cliente realiza un pago, se publica un evento en Kafka para que otros sistemas (riesgo, fraude) lo procesen.
 - **Desacoplamiento:** Permite que sistemas lentos (legacy) reciban datos sin afectar la respuesta al usuario.
- **Flujo típico:**
 1. La app móvil envía una solicitud de pago → API Gateway → Core Digital.
 2. El Core Digital publica un evento "PagoRealizado" en Kafka.
 3. Sistemas suscritos (ej: antifraude) consumen el evento para análisis en segundo plano.

2.4. Autenticación Centralizada (Keycloak)

- **Funcionamiento:**
 - Los usuarios inician sesión una vez y reciben un token JWT.
 - Cada solicitud posterior incluye el token, que el API Gateway valida antes de permitir acceso.
- **Ventajas:**
 - Evita que cada servicio implemente su propio login.
 - Permite revocar accesos rápidamente (ej: si un empleado deja el banco).

3. Flujos de Integración Detallados

3.1. Ejemplo: Proceso de Transferencia Bancaria

1. **Paso 1 (Frontend):**
 - El cliente ingresa los datos en la app móvil → Solicitud HTTPS al API Gateway.
2. **Paso 2 (Seguridad):**
 - Kong verifica el token JWT con Keycloak. Si es válido, permite el paso.
3. **Paso 3 (Orquestación):**
 - MuleSoft recibe la solicitud y:
 - a) Consulta el saldo (SOAP al mainframe).
 - b) Valida límites (REST al core digital).
 - c) Ejecuta la transferencia (gRPC al servicio de pagos).
4. **Paso 4 (Eventos):**
 - El core digital publica "TransferenciaExitosa" en Kafka → Sistemas de riesgo y fraude lo consumen para análisis.
5. **Paso 5 (Respuesta):**

- MuleSoft consolida las respuestas y envía un JSON uniforme a la app móvil.

3.2. Integración con Terceros

- **Proveedores externos** (ej: procesadores de pagos) se conectan vía APIs REST gestionadas por Kong.
- **Transformación de formatos:** MuleSoft adapta APIs externas (ej: ISO8583) al formato interno del banco.

4. Tecnologías de Soporte

4.1. Almacenamiento y Caché

- **Redis:** Cachea respuestas frecuentes (ej: saldos de cuentas) para reducir latencia.
- **Bases de datos:**
 - **PostgreSQL:** Almacena datos del nuevo core digital.
 - **Oracle:** Soporta el legacy (en proceso de migración).

4.2. Monitoreo (Prometheus + Grafana)

- **Métricas clave:**
 - Tiempos de respuesta por servicio (legacy vs. digital).
 - Errores en transformaciones de datos (MuleSoft).
 - Uso del bus de eventos (Kafka).
- **Alertas:** Notifican si un servicio legacy tarda más de lo esperado.

5. Conclusiones Técnicas

5.1. Ventajas de la Arquitectura

- **Interoperabilidad:** Convive con lo antiguo (SOAP) y lo moderno (REST/gRPC).

- **Escalabilidad:** Kafka y microservicios permiten añadir nuevos componentes sin rehacer el sistema.
- **Seguridad:** Keycloak + Kong centralizan la gestión de accesos.

5.2. Desafíos

- **Latencia en legacy:** Las llamadas a mainframes pueden ralentizar procesos (se mitiga con caché y eventos asíncronos).
- **Complejidad operativa:** Requiere equipos especializados en MuleSoft, Kafka, y migración de datos.

Nota final: Esta arquitectura es un modelo para bancos en transición digital, donde la capa de integración (MuleSoft + Kafka) es el "pegamento" que une sistemas dispares sin reemplazarlos abruptamente.